Can you explain the different Transformation you've done in your project?

PySpark

Be Prepared Learn 50 Pyspark Transformation to Stand Out

Abhishek Agrawal
Azure Data Engineer

1. Normalization

Scaling data to a range between 0 and 1.

```
from pyspark.ml.feature import MinMaxScaler
scaler = MinMaxScaler(inputCol="features", outputCol="scaled_features")
scaled_data = scaler.fit(data).transform(data)
```

2. Standardization

Transforming data to have zero mean and unit variance

```
from pyspark.ml.feature import StandardScaler
scaler = StandardScaler(inputCol="features", outputCol="scaled_features")
scaled_data = scaler.fit(data).transform(data)
```

3. Log Transformation

Applying a logarithmic transformation to handle skewed data.

```
from pyspark.ml.feature import StandardScaler
scaler = StandardScaler(inputCol="features", outputCol="scaled_features")
scaled_data = sfrom pyspark.ml.feature import StandardScaler

# Initialize the StandardScaler
scaler = StandardScaler(
    inputCol="features",
    outputCol="scaled_features"
)

# Fit the scaler to the dataset and transform the data
scaled_data = scaler.fit(data).transform(data)
caler.fit(data).transform(data)
```

4. Binning

Grouping continuous values into discrete bins.

```
from pyspark.sql.functions import when

# Add a new column 'bin_column' based on conditions
data = data.withColumn(
    "bin_column",
    when(data["value"] < 10, "Low")
    .when(data["value"] < 20, "Medium")
    .otherwise("High")
)
```

5. One-Hot Encoding

Converting categorical variables into binary columns.

```
from pyspark.ml.feature import OneHotEncoder, StringIndexer

# Step 1: Indexing the categorical column
indexer = StringIndexer(inputCol="category", outputCol="category_index")
indexed_data = indexer.fit(data).transform(data)

# Step 2: One-hot encoding the indexed column
encoder = OneHotEncoder(inputCol="category_index", outputCol="category_onehot")
encoded_data = encoder.fit(indexed_data).transform(indexed_data)
```

6. Label Encoding

Converting categorical values into integer labels.

```
from pyspark.ml.feature import StringIndexer

# Step 1: Create a StringIndexer to index the 'category' column
indexer = StringIndexer(inputCol="category", outputCol="category_index")

# Step 2: Fit the indexer on the data and transform it
indexed_data = indexer.fit(data).transform(data)
```

7. Pivoting

Pivoting is the process of transforming long-format data (where each row represents a single observation or record) into wide-format data (where each column represents a different attribute or category). This transformation is typically used when you want to turn a categorical variable into columns and aggregate values accordingly.

```
# Pivoting the data to create a summary of sales by month for each ID
pivoted_data = data.groupBy("id") \
    .pivot("month") \
    .agg({"sales": "sum"})
= data.groupBy("id").pivot("month").agg({"sales": "sum"})
```

8. Unpivoting

Unpivoting is the opposite of pivoting. It transforms wide-format data (where each column represents a different category or attribute) into long-format data (where each row represents a single observation). This is useful when you want to turn column headers back into values.

```
# Unpivoting the data to convert columns into rows
unpivoted_data = data.selectExpr(
    "id",
    "stack(2, 'Jan', Jan, 'Feb', Feb) as (month, sales)"
)
```

9. Aggregation

Summarizing data by applying functions like sum(), avg(), etc.

```
# Aggregating data by category to compute the sum of values
aggregated_data = data.groupBy("category") \
    .agg({"value": "sum"})
```

10. Feature Extraction

Extracting useful features from raw data.

```
from pyspark.sql.functions import year, month, dayofmonth

# Add year, month, and day columns to the DataFrame
data = (
    data
    .withColumn("year", year(data["timestamp"]))
    .withColumn("month", month(data["timestamp"]))
    .withColumn("day", dayofmonth(data["timestamp"]))
)
```

11. Outlier Removal

Filtering out extreme values (outliers).

```
# Filter rows where the 'value' column is less than 1000
filtered_data = data.filter(data["value"] < 1000)
```

12. Data Imputation

Filling missing values with the mean or median.

```
from pyspark.ml.feature import Imputer

# Create an Imputer instance
imputer = Imputer(inputCols=["column"], outputCols=["imputed_column"])

# Fit the imputer model and transform the data
imputed_data = imputer.fit(data).transform(data)
```

13. Date/Time Parsing

Converting string to datetime objects.

```
from pyspark.sql.functions import to_timestamp

# Convert the 'date_string' column to a timestamp with the specified format
data = data.withColumn("timestamp", to_timestamp(data["date_string"], "yyyy-MM-dd"))
```

14. Text Transformation

Converting text to lowercase.

```
from pyspark.sql.functions import lower

# Convert the text in 'text_column' to lowercase and store it in a new column
data = data.withColumn("lowercase_text", lower(data["text_column"]))
```

15. Data Merging

Combining two datasets based on a common column.

```
# Perform an inner join between data1 and data2 on the 'id' column
merged_data = data1.join(data2, data1["id"] == data2["id"], "inner")
```

16. Data Joining

Joining data using inner, left, or right joins.

```
# Perform a left join between data1 and data2 on the 'id' column
joined_data = data1.join(data2, on="id", how="left")
```

17. Filtering Rows

Filtering rows based on a condition.

```
# Filter rows where the 'value' column is greater than 10
filtered_data = data.filter(data["value"] > 10)
```

18. Column Renaming

Renaming columns for clarity.

```
# Rename the column 'old_column' to 'new_column'
data = data.withColumnRenamed("old_column", "new_column")
```

19. Column Dropping

Removing unnecessary columns.

```
# Drop the 'unwanted_column' from the DataFrame
data = data.drop("unwanted_column")
```

20. Column Conversion

Converting a column from one data type to another.

```
from pyspark.sql.functions import col

# Convert 'column_string' to an integer and create a new column 'column_int'
data = data.withColumn("column_int", col("column_string").cast("int"))
```

21. Type Casting

Changing the type of a column (e.g., from string to integer).

```
# Convert 'column_string' to an integer and create a new column 'column_int'
data = data.withColumn("column_int", data["column_string"].cast("int"))
```

22. Duplicate Removal

Removing duplicate rows based on specified columns.

```
# Remove duplicate rows based on 'column1' and 'column2'
data = data.dropDuplicates(["column1", "column2"])
```

23. Null Value Removal

Filtering rows with null values in specified columns.

```
# Filter rows where the 'column' is not null
cleaned_data = data.filter(data["column"].isNotNull())
```

24. Windowing Functions

Using window functions to rank or aggregate data.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import rank

# Define a window specification partitioned by 'category' and ordered by 'value'
window_spec = Window.partitionBy("category").orderBy("value")

# Add a 'rank' column based on the window specification
data = data.withColumn("rank", rank().over(window_spec))
```

25. Column Combination

Combining multiple columns into one.

```
from pyspark.sql.functions import concat

# Concatenate 'first_name' and 'last_name' columns to create 'full_name'
data = data.withColumn("full_name", concat(data["first_name"], data["last_name"])
)
```

26. Cumulative Sum

Calculating a running total of a column.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum

# Define a window specification ordered by 'date' with an unbounded
preceding frame
window_spec = Window.orderBy("date").
rowsBetween(Window.unboundedPreceding, Window.currentRow)

# Add a 'cumulative_sum' column that computes the cumulative sum of 'value'
data = data.withColumn("cumulative_sum", sum("value").over(window_spec))
```

27. Rolling Average

Calculating a moving average over a window of rows.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import avg

window_spec = Window.orderBy("date").rowsBetween(-2, 2)

data = data.withColumn("rolling_avg", avg("value").over(window_spec))
```

28. Value Mapping

Mapping values of a column to new values.

```
from pyspark.sql.functions import when

# Map 'value' column: set 'mapped_column' to 'A' if 'value' is 1, otherwise 'B'
data = data.withColumn("mapped_column", when(data["value"] == 1, "A").
otherwise("B"))
```

29. Subsetting Columns

Calculating a moving average over a window of rows.

```
Selecting only a subset of columns from the dataset.
```

30. Column Operations

Performing arithmetic operations on columns.

```
# Create a new column 'new_column' as the sum of 'value1' and 'value2'
data = data.withColumn("new_column", data["value1"] + data["value2"])
```

31. String Splitting

Splitting a string column into multiple columns based on a delimiter.

```
from pyspark.sql.functions import split

# Split the values in 'column' by a comma and store the result in 'split_column'
data = data.withColumn("split_column", split(data["column"], ","))
```

32. Data Flattening

Flattening nested structures (e.g., JSON) into a tabular format.

```
from pyspark.sql.functions import explode

# Flatten the array or map in 'nested_column' into multiple rows in
'flattened_column'
data = data.withColumn("flattened_column", explode(data["nested_column"]))
```

33. Sampling Data

Taking a random sample of the data.

```
# Sample 10% of the data
sampled_data = data.sample(fraction=0.1)
```

34. Stripping Whitespace

Removing leading and trailing whitespace from string columns.

```
from pyspark.sql.functions import trim

# Remove leading and trailing spaces from 'string_column' and create
'trimmed_column'
data = data.withColumn("trimmed_column", trim(data["string_column"]))
```

35. String Replacing

Replacing substrings within a string column.

```
from pyspark.sql.functions import regexp_replace

# Replace occurrences of 'old_value' with 'new_value' in 'text_column' and
create 'updated_column'
data = data.withColumn("updated_column", regexp_replace(data["text_column"],
"old_value", "new_value"))
```

36. Date Difference

Calculating the difference between two date columns.

```
from pyspark.sql.functions import datediff

# Calculate the difference in days between 'end_date' and 'start_date', and
create 'date_diff' column
data = data.withColumn("date_diff", datediff(data["end_date"],
data["start_date"]))
```

37. Window Rank

Ranking rows based on a specific column.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import rank

# Define a window specification ordered by 'value'
window_spec = Window.orderBy("value")

# Add a 'rank' column based on the window specification
data = data.withColumn("rank", rank().over(window_spec))
```

38. Multi-Column Aggregation

Performing multiple aggregation operations on different columns.

```
# Group by 'category' and calculate the sum of 'value1' and the average of
'value2'
aggregated_data = data.groupBy("category").agg(
    {"value1": "sum", "value2": "avg"}
)
```

39. Date Truncation

Truncating a date column to a specific unit (e.g., year, month).

```
from pyspark.sql.functions import trunc

# Truncate the date_column to the beginning of the month and add as a new
column
data = data.withColumn("truncated_date", trunc(data["date_column"], "MM"))
```

40. Repartitioning Data

Changing the number of partitions for better performance

```
# Repartition the DataFrame into 4 partitions
data = data.repartition(4)
```

41. Adding Sequence Numbers

Assigning a unique sequence number to each row.

```
from pyspark.sql.functions import monotonically_increasing_id

# Add a new column 'row_id' with a unique, monotonically increasing ID
data = data.withColumn("row_id", monotonically_increasing_id())
```

42. Shuffling Data

Randomly shuffling rows in a dataset.

```
from pyspark.sql.functions import rand

# Shuffle the DataFrame by ordering rows randomly
shuffled_data = data.orderBy(rand())
```

43. Array Aggregation

Combining values into an array.

```
from pyspark.sql.functions import collect_list

# Group by 'id' and aggregate 'value' into a list, storing it in a new column
'values_array'
data = data.groupBy("id").agg(collect_list("value").alias("values_array"))
```

44. Scaling

Scaling features by a specific factor.

```
from pyspark.ml.feature import QuantileDiscretizer

# Initialize the QuantileDiscretizer with input column, output column, and
number of buckets
scaler = QuantileDiscretizer(inputCol="value", outputCol="scaled_value",
numBuckets=10)

# Fit the discretizer to the data and transform the DataFrame
scaled_data = scaler.fit(data).transform(data)
```

45. Bucketing

Grouping continuous data into buckets.

```
from pyspark.ml.feature import Bucketizer

# Define split points for bucketing
splits = [0, 10, 20, 30, 40, 50]

# Initialize the Bucketizer with splits, input column, and output column
bucketizer = Bucketizer(splits=splits, inputCol="value",
outputCol="bucketed_value")

# Apply the bucketizer transformation to the DataFrame
bucketed_data = bucketizer.transform(data)
```

46. Boolean Operations

Performing boolean operations on columns.

```
from pyspark.sql.functions import col

# Add a new column 'is_valid' indicating whether the 'value' column is greater than 10
data = data.withColumn("is_valid", col("value") > 10)
```

47. Extracting Substrings

Extracting a portion of a string from a column.

```
from pyspark.sql.functions import substring

# Add a new column 'substring' containing the first 5 characters of 'text_column'
data = data.withColumn("substring", substring(col("text_column"), 1, 5))
```

48. JSON Parsing

Parsing JSON data into structured columns.

```
from pyspark.sql.functions import from_json

# Parse the JSON data in the 'json_column' into a structured column 'json_data' using the specified schema
data = data.withColumn("json_data", from_json(col("json_column"), schema))
```

49. String Length

Finding the length of a string column

```
from pyspark.sql.functions import length

# Add a new column 'string_length' containing the length of the strings in
'text_column'
data = data.withColumn("string_length", length(col("text_column")))
```

50. Row-wise Operations

Applying row-wise functions to a dataset by applying a custom function to a column using a User-Defined Function (UDF).

```
from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType

# Define a function to add 2 to the input value
def add_two(value):
    return value + 2

# Register the function as a UDF
add_two_udf = udf(add_two, IntegerType())

# Apply the UDF to the 'value' column and create a new column
'incremented_value'
data = data.withColumn("incremented_value", add_two_udf(col("value")))
```

Follow for more content like this

Abhishek Agrawal

Azure Data Engineer